# DATA 266 — Homework 7

Sneha Singh
SID4 = 0670 | SEED = 670 | SLICE = 670 | HP_ID = 4 | CLS_A = 0 | CLS_B = 5

Notebook: `autoencoders_fashionmnist.ipynb`. Dataset: FashionMNIST.

Same SID/SEED as HW1. No extra HP_ID network. The two trained models are a fully connected autoencoder (FC-AE) and a VAE.

## 1. GANs

### 1.1 Architecture

A GAN is two networks trained against each other. The generator G takes random noise z and turns it into a fake sample. The discriminator D looks at a sample and says how likely it is to be real. The game is written as:

$$\min_G \max_D \; \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

Training goes in turns. First G is frozen, and D gets a batch of real data and a batch of G's fakes and learns to score the real ones high and the fakes low. Then D is frozen and G is updated so that D scores its fakes higher. One detail: G usually maximizes $\log D(G(z))$ instead of minimizing $\log(1 - D(G(z)))$. Early on D wins easily, and with the original form G gets almost no gradient.

The goal is for the generator's distribution $p_g$ to end up equal to the data distribution $p_{data}$. If G is fixed, the best D is $p_{data}(x) / (p_{data}(x) + p_g(x))$. Plug that into the game and G is minimizing $2 \cdot \mathrm{JSD}(p_{data} \,\|\, p_g) - \log 4$, which bottoms out when $p_g = p_{data}$. At that point D can only guess, so it outputs 1/2 everywhere. In practice you rarely get there. Both networks are non-convex and update at the same time, so the losses tend to bounce around instead of settling.

### 1.2 Mode collapse

Mode collapse is when G keeps producing only a few kinds of output (a couple of digits out of ten, say) even though the data has many. Lots of different z end up mapped to nearly the same image.

It happens because D judges each sample on its own, so nothing in the loss pays G for variety. G finds whatever D likes most right now and keeps making that. D then learns to reject it, G hops to another spot D likes, and it keeps cycling instead of covering everything. It gets worse when D is too strong or the real and fake distributions do not overlap, because the original loss then gives G almost no gradient.

Two things that help:

1. **WGAN-GP.** D becomes a critic that estimates the Wasserstein distance, with no sigmoid or log. It has to be 1-Lipschitz, and a gradient penalty $\lambda(\|\nabla D(\hat{x})\| - 1)^2$ on points between real and fake enforces that. This distance still gives a usable gradient when the distributions do not overlap, so the critic can be trained well without G's gradient dying, and G has no single sharp peak to chase.
2. **Minibatch discrimination.** D gets extra features that compare each sample with the others in the same batch. If G makes a batch of near-identical images, D can spot it easily, so G is pushed to vary within a batch.

### 1.3 Three variants

**DCGAN** (Radford et al., 2015). Early GANs used fully connected layers and were unstable on images. DCGAN swapped in strided convolutions instead of pooling, transposed convolutions in G, batch norm in both, ReLU in G (Tanh at the output), LeakyReLU in D, and no fully connected hidden layers. That made image GANs stable enough to be practical.

**Conditional GAN** (Mirza & Osindero, 2014). A plain GAN gives you no say in what it makes. A conditional GAN feeds a label y into both G and D (a class, or in pix2pix an input image). D then checks whether the sample is real for that label, so G has to make the class you asked for. Pix2pix uses this with paired images and adds an L1 loss and a patch-based D.

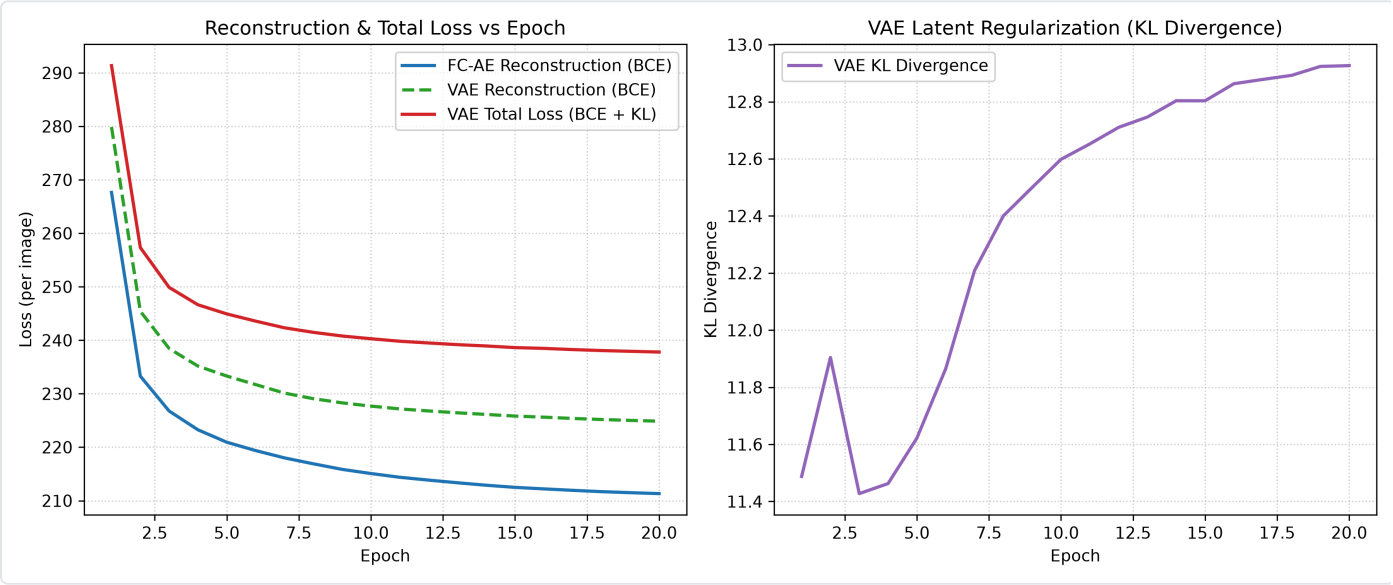
**CycleGAN** (Zhu et al., 2017). Image-to-image translation needed paired examples, and those are often not available. CycleGAN trains two generators (X to Y and Y to X) and two discriminators, plus a cycle-consistency loss: F(G(x)) ≈ x and G(F(y)) ≈ y. That ties the output to the input's content, so it works on unpaired sets like horses and zebras.

## 2. FC-AE vs VAE

60,000 train and 10,000 test images, 28×28, pixels 0 to 1. Both models use 784-512-128-20 for the encoder and the mirror for the decoder, sigmoid on the output. Adam, lr 1e-3, batch 128, 20 epochs, on MPS. Loss is BCE summed over the pixels and averaged per image. The VAE has μ and log σ² heads, samples $z = \mu + \sigma \cdot \varepsilon$, and adds the KL term to N(0, I) with weight 1.

| | FC-AE | VAE |
|---|---|---|
| trainable params | 941,092 | 943,672 |
| train loss, epoch 1 | 267.6440 | 291.3547 |
| train loss, epoch 20 | 211.3226 | 237.7973 |

The VAE loss is BCE + KL. At epoch 20 that is 224.8707 + 12.9265.



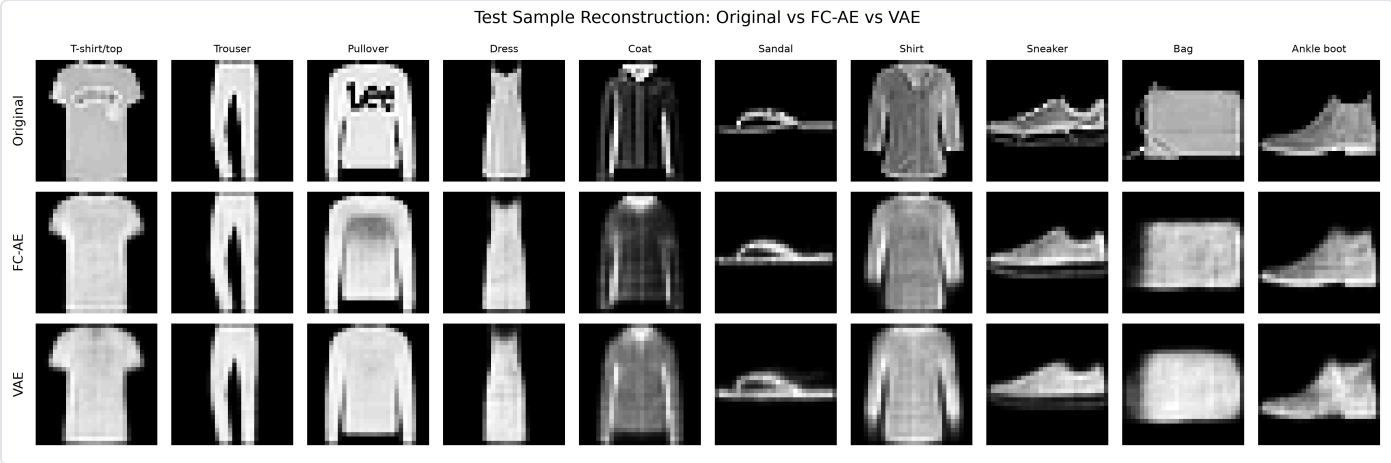
### Test set

| | FC-AE | VAE |
|---|---|---|
| Recon BCE per image | 213.6070 | 226.8435 |
| Recon MSE per pixel | 0.009739 | 0.015663 |
| KL per image | none | 13.0439 |
| BCE + KL | 213.6070 | 239.8874 |

BCE + KL is not a fair comparison, since the FC-AE has no KL term.

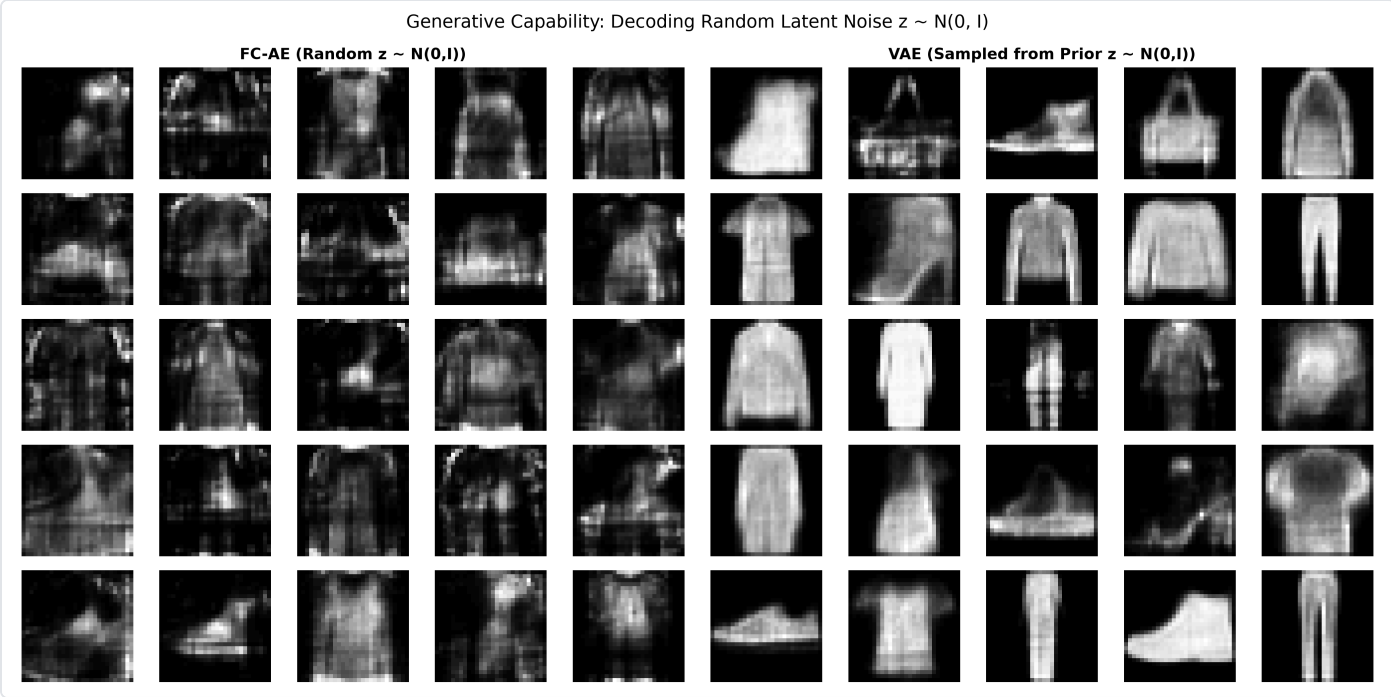
### Reconstruction

FC-AE reconstructs better. Nothing pulls its code toward a prior, so all 20 dimensions go to reconstruction. In the figure the VAE copies are blurrier, especially the bag and the ankle boot. Both lose the print on the pullover. Rows are original, FC-AE, VAE.



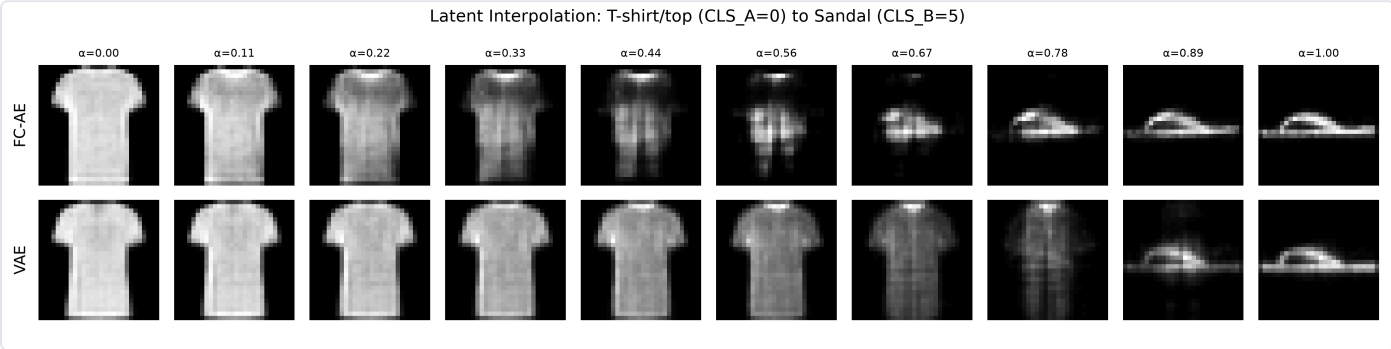

### Random z

I decode z ~ N(0, I) in both. The FC-AE gives smudges and stripes, and almost none look like clothing. The VAE gives sneakers, boots, trousers, a dress, shirts and a bag, though some are blurry or mixed. The KL term pulls the VAE codes toward N(0, I), so a random z lands where the decoder was trained. The FC-AE has no reason to use that region.



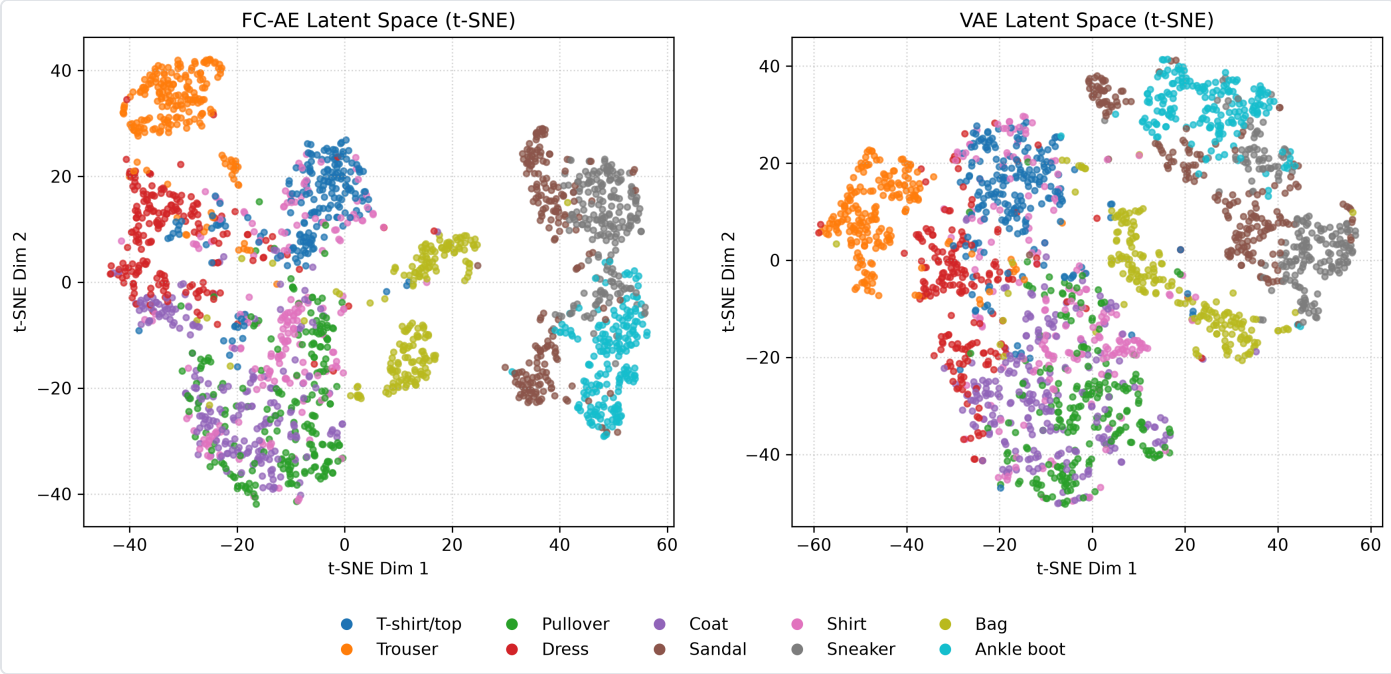

### Interpolation, class 0 to class 5

First T-shirt and first sandal in the test set, 10 even steps in latent space (VAE uses μ). The top row is the FC-AE. It fades slowly: the middle frames are a smudge of shirt and sandal, and it is a sandal by about α=0.8. The bottom row is the VAE. It stays shirt-shaped and fades until about α=0.8, then is a sandal by α=0.9. I did not see a clean in-between item in either row. This is one pair, so I would not claim more.




### t-SNE

2,000 test images, 20-d codes. Both plots separate trousers, bags and the shoe classes. T-shirt, pullover, coat and shirt overlap in both. I cannot see a clear difference between the two, so I am not claiming the VAE clusters are better.




### Which one

For compressing and rebuilding images the FC-AE is better (MSE 0.0097 vs 0.0157). For making new images the VAE is the one that works, because its codes match the prior I sample from.